Homework 4 - CSCI633

Evan Lin

04/25/2026

Question 1. Binary PSO and APSO

a) Why and how would your algorithm different from standard PSO (Particle Swarm Optimization)?

Intuition: Standard PSO works with continuous positions. For binary optimization, we cannot simply add velocities. Instead, we interpret the velocity as a probability that a bit becomes 1, using a sigmoid function to map velocities into probabilities.

Modifications:

- Velocities remain real numbers and are updated using the standard PSO.
- Each velocity component v_{ik} is passed through a sigmoid function:

$$S(v_{ik}) = \frac{1}{1 + e^{-v_{ik}}}$$

  to give the probability that the corresponding position bit is 1.

- The binary position is set by a random threshold:

$$x_{ik} = \begin{cases} 1, & \text{if } \text{rand}(0,1) < S(v_{ik}) \\ 0, & \text{otherwise} \end{cases}$$

- Personal best $\mathbf{p}_i$ and global best $\mathbf{g}$ are stored as binary vectors.

This creates Binary Particle Swarm Optimization (BPSO). It differs from standard PSO because positions are discrete, velocities control probabilities rather than displacements, and a sigmoid transformation plus random draw is used at each update.

b) Formally present your binary PSO process as an algorithm.

Algorithm 1 Binary Particle Swarm Optimization (BPSO)

Input: objective function f, swarm size n, dimensions d, inertia w, coefficients c_1, c_2, max iterations $T_{\max}$
Output: global best position $\mathbf{g}$
Initialize:
for each particle $i = 1$ to n **do**
  for each dimension $k = 1$ to d **do**
    $x_{ik} \leftarrow \text{random}\{0, 1\}$
    $v_{ik} \leftarrow \text{random}[-V_{\max}, V_{\max}]$
  end for
  $\mathbf{p}_i \leftarrow \mathbf{x}_i$ ▷ personal best
  $f_i \leftarrow f(\mathbf{x}_i)$
end for
$\mathbf{g} \leftarrow \arg\min_i f_i$ ▷ global best
for $t = 1$ to $T_{\max}$ **do**
  for each particle i **do**
    for each dimension k **do**
      $r_1, r_2 \leftarrow \text{random}[0, 1]$
      $v_{ik} \leftarrow w v_{ik} + c_1 r_1 (p_{ik} - x_{ik}) + c_2 r_2 (g_k - x_{ik})$
      $S(v_{ik}) \leftarrow \dfrac{1}{1 + e^{-v_{ik}}}$
      $x_{ik} \leftarrow \begin{cases} 1 & \text{if random}[0, 1] < S(v_{ik}) \\ 0 & \text{otherwise} \end{cases}$
    end for
    $f_{\text{new}} \leftarrow f(\mathbf{x}_i)$
    if $f_{\text{new}} \leq f_i$ **then**
      $\mathbf{p}_i \leftarrow \mathbf{x}_i$, $f_i \leftarrow f_{\text{new}}$
      if $f_{\text{new}} \leq f(\mathbf{g})$ **then**
        $\mathbf{g} \leftarrow \mathbf{x}_i$
      end if
    end if
  end for
end for
return $\mathbf{g}$

c) Repeat "part b" for accelerated PSO (APSO)

Algorithm 2 Binary Accelerated Particle Swarm Optimization (binary APSO)

Input: objective function f, swarm size n, dimensions d, acceleration coefficient β, initial randomness α_0, cooling rate γ, max iterations $T_{\max}$
Output: global best position $\mathbf{g}$
Initialize:
for each particle $i = 1$ to n **do**
    for each dimension $k = 1$ to d **do**
        $x_{ik} \leftarrow \text{random}\{0, 1\}$
        $v_{ik} \leftarrow 0$
    end for
    $f_i \leftarrow f(\mathbf{x}_i)$
end for
$\mathbf{g} \leftarrow \arg\min_i f_i$ ▷ global best
for $t = 1$ to $T_{\max}$ **do**
    $\alpha \leftarrow \alpha_0 \cdot \gamma^t$ ▷ decrease randomness
    for each particle i **do**
        for each dimension k **do**
            $\epsilon \leftarrow \text{random}[-1, 1]$
            $v_{ik} \leftarrow v_{ik} + \beta\,(g_k - x_{ik}) + \alpha\,\epsilon$ ▷ APSO velocity update
            $S(v_{ik}) \leftarrow \dfrac{1}{1 + e^{-v_{ik}}}$
            $x_{ik} \leftarrow \begin{cases} 1 & \text{if random}[0, 1] < S(v_{ik}) \\ 0 & \text{otherwise} \end{cases}$
        end for
        $f_{\text{new}} \leftarrow f(\mathbf{x}_i)$
        if $f_{\text{new}} \leq f(\mathbf{g})$ **then**
            $\mathbf{g} \leftarrow \mathbf{x}_i$
        end if
    end for
end for
return g

d) Explain one limitation of your binary PSO/APSO.

One major limitation of binary PSO and binary APSO is premature convergence to suboptimal solutions. Because the binary search space is discrete and the sigmoid function maps velocities to probabilities, particles can quickly lose diversity and become trapped in local optima. Once all bits in the swarm become similar (all 0 or all 1), it is difficult to escape without sufficient randomization.

Question 2. Firefly Algorithm (FA)

(a) Four-peak function

Population size	**Set A** ($\beta_0 = 1, \gamma = 1, \alpha = 0.5$)	**Set B** ($\beta_0 = 0.8, \gamma = 0.5, \alpha = 0.2$)
5	1.99885798 ± 0.00147801	1.99974176 ± 0.00039539
10	1.99984882 ± 0.00020656	1.99996690 ± 0.00004263
25	1.99997255 ± 0.00002791	1.99999524 ± 0.00000561
50	1.99999258 ± 0.00000599	1.99999914 ± 0.00000106
100	1.99999844 ± 0.00000166	2.00000008 ± 0.00000015

Discussion: FA consistently finds the global maximum (2.0). Larger populations improve both accuracy and precision. Set B (lower α and γ) performs better for small swarms because reduced noise and longer attraction range help convergence.

(b) Eggcrate function

Population size	**Set A** ($\beta_0 = 1, \gamma = 1, \alpha = 0.5$)	**Set B** ($\beta_0 = 0.8, \gamma = 0.5, \alpha = 0.2$)
5	11.46695559 ± 7.36723617	11.87848508 ± 6.01023748
10	6.23216614 ± 3.93790365	7.54290313 ± 3.62805583
25	4.83897275 ± 4.08998481	4.80847243 ± 4.60788924
50	0.88558464 ± 2.54161134	1.11450798 ± 2.89717546
100	0.00217418 ± 0.00341868	0.00051179 ± 0.00110636

Discussion: The eggcrate function has many local minima. With small populations ($n = 5, 10, 25$), the algorithm often gets trapped in local minima, resulting in mean values far from the global optimum (0) and large standard deviations. As the swarm size increases to 50, performance improves dramatically. With population size set to 100, both hyperparameter sets find solutions extremely close to zero. Set B gives slightly better precision, showing that reducing random noise and increasing the attraction range helps the swarm stabilize.